

--Jose Francisco Gutierrez Villalobos

--Felipe Rodriguez Matarrita

--A1

CREATE TABLE Pedido

(

id_pedido INT NOT NULL,

id_cliente INT NOT NULL,

fecha DATE DEFAULT GETDATE(),

total DECIMAL(8,2) CHECK(total > 0),

estado VARCHAR(20) NOT NULL DEFAULT 'Pendiente' CHECK (estado IN ('Pendiente','Enviado','Cancelado')), --habia error aqui en el parentesis en check.

PRIMARY KEY (id_pedido),

FOREIGN KEY (id_cliente) REFERENCES Cliente(id) -- la llave primaria estaba mal diseñada por que era una llave primaria compuesta

);

--A2

--La intencion de la consulta es obtener los socios con mas de un prestao actualmente activo, no devuelto, encuentra todos los errores.

SELECT s.nombre, s.apellido, COUNT(p.id_prestamo) AS total_prestamos

FROM Socio s

JOIN Prestamo p ON s.id_socio = p.id_socio

WHERE p.fecha_devolucion_real IS NULL -- estaba mal usar = NULL y el nombre de la columna

GROUP BY s.nombre, s.apellido -- faltaba agrupar por apellido

HAVING COUNT(p.id_prestamo) > 1;

--A3

-- procedimiento almacenado

--Este procedimiento registra la devolucion de un libro identifica los problemas de robustez y correccion

--¿que ocurre si el prestamo no existe o ya fue devuelto?

--¿que garantia de integridad falta? reescribelo correctamente.

--A3

-- ESO NO LO HABIAMOS VISTO, EL PROFE LO QUITO DEL EXAMEN :)

--B1

--Instruccion: Responde con texto, sin escribir codigo SQL completo. Se valora la precision del razonamiento no la longitud de la respuesta

--¿Cual es la diferencia en el resultado de estas dos consultas? Da un ejemplo concreto de cuando cada una devolveria filas distintas

--consulta X

SELECT L.titulo, C.nombre

FROM libro L

INNER JOIN categoria C ON L.id_categoria = C.id_categoria;

--consulta Y

SELECT L.titulo, C.nombre

FROM libro L

LEFT JOIN categoria C ON L.id_categoria = C.id_categoria;

-- Respuesta:

--INNER JOIN solo muestra libros con categoría, LEFT JOIN muestra todos los libros.

--Ejemplo si un libro no tiene categoría, en INNER no aparece y en LEFT sí

--B2

--Un compañero escribe la siguiente consulta para obtener el numero de prestamos por socio y muestra solo los que tienen mas de 2

```
SELECT id_socio, COUNT(*) AS total  
FROM prestamo  
WHERE COUNT(*) > 2  
GROUP BY id_socio;
```

--La consulta falla, Explica por que y describe como deberia escribirse correctamente

--(no es necesario escribir el codigo completo, solo explicar el concepto)

--Respuesta B2:

--Falla porque no se puede usar COUNT(*) en WHERE, las agregaciones se filtran con HAVING despues del group by

--B3

--Explica la diferencia entre DELETE, DROP en SQL server. ¿En que situacion usarias cada uno?

--¿Cual de ellos puede revertirse con un ROLLBACK dentro de una transaccion explicita?

--respuesta B3:

--DELETE borra filas, DROP elimina la tabla completa. DELETE se usa para datos,

--DROP para quitar la tabla. DELETE se puede revertir con el ROLLBACK

--B4

--¿Que significa que una columna tenga el constraint NOT NULL?

--¿Es suficiente NOT NULL para que no haya valores duplicados en esa columna?

--justifique su respuesta con un ejemplo.

-- Respuesta B4:

--NOT NULL significa que no puede ser NULL no evita duplicados

--Ejemplo: varios registros pueden tener el mismo email, para eso se usa UNIQUE

--B5

--Analiza esta sentencia

```
SELECT titulo, AVG(num_paginas) AS promedio_paginas
```

```
FROM libro
```

```
GROUP BY id_categoria;
```

--¿Produce un error o devuelve resultados? Explica exactamente que esta mal (si hay algo) y como arreglarlo.

--Respuesta B5:

--Produce error, porque titulo no esta en el GROUP BY ni en una funcion de agregacion.

--Para arreglarlo, se agrega titulo al GROUP BY o se quita del SELECT

----- seccio c crear base de datos -----

```
Create database filmoteca
```

```
go
```

```
use filmoteca
```

```
go
```

```
CREATE TABLE Pelicula
```

```
(
```

```
PeliculaID INT IDENTITY(1,1) PRIMARY KEY,
```

```
Titulo VARCHAR(150) NOT NULL,
```

```
AnioEstreno INT NOT NULL,
```

```
DuracionMin INT NOT NULL,
```

```
Genero VARCHAR(50) NOT NULL,
```

```
CONSTRAINT CHK_Pelicula_Genero CHECK (Genero IN  
( 'Accion','Drama','Comedia','Documental','Ciencia Ficción','Terror','Otro'))
```

```
);
```

```
CREATE TABLE Director
```

```
(
```

```
DirectorID INT IDENTITY(1,1) PRIMARY KEY,
```

```
Nombre VARCHAR(100) NOT NULL,
```

```
Apellido VARCHAR(100) NOT NULL,
```

```
PaisOrigen VARCHAR(100) NOT NULL
```

```
);
```

```
CREATE TABLE PeliculaDirector
(
    PeliculaID INT NOT NULL,
    DirectorID INT NOT NULL,
    CONSTRAINT PK_PeliculaDirector PRIMARY KEY (PeliculaID, DirectorID),
    CONSTRAINT FK_PD_Pelicula FOREIGN KEY (PeliculaID) REFERENCES
    Pelicula(PeliculaID),
    CONSTRAINT FK_PD_Director FOREIGN KEY (DirectorID) REFERENCES
    Director(DirectorID)
);
```

```
CREATE TABLE Copia
(
    CopiaID INT IDENTITY(1,1) PRIMARY KEY,
    PeliculaID INT NOT NULL,
    Formato VARCHAR(50) NOT NULL,
    Estado VARCHAR(50) NOT NULL DEFAULT 'Disponible',
    CONSTRAINT FK_Copia_Pelicula FOREIGN KEY (PeliculaID) REFERENCES
    Pelicula(PeliculaID),
    CONSTRAINT CHK_Copia_Formato CHECK (Formato IN ('DVD','Blu-
    ray','Digital')),
    CONSTRAINT CHK_Copia_Estado CHECK (Estado IN
    ('Disponible','Prestado','Deteriorado'))
);
```

```
CREATE TABLE Miembro
(
    MiembroID INT IDENTITY(1,1) PRIMARY KEY,
```

```
Nombre VARCHAR(100) NOT NULL,  
Apellido VARCHAR(100) NOT NULL,  
Email VARCHAR(150) NOT NULL UNIQUE,  
Tipo VARCHAR(50) NOT NULL,  
Activo BIT NOT NULL DEFAULT 1,  
CONSTRAINT CHK_Miembro_Tipo CHECK (Tipo IN ('Estudiante','Profesor'))  
);
```

```
CREATE TABLE Prestamo  
(  
PrestamoID INT IDENTITY(1,1) PRIMARY KEY,  
CopiaID INT NOT NULL,  
MiembroID INT NOT NULL,  
FechaPrestamo DATE NOT NULL DEFAULT GETDATE(),  
FechaDevolucionPrevista DATE NOT NULL,  
FechaDevolucionReal DATE NULL,  
CONSTRAINT FK_Prestamo_Copia FOREIGN KEY (CopiaID) REFERENCES  
Copia(CopiaID),  
CONSTRAINT FK_Prestamo_Miembro FOREIGN KEY (MiembroID)  
REFERENCES Miembro(MiembroID)  
);
```

```
--a
```

```
ALTER TABLE Pelicula  
ADD Puntuacion DECIMAL(3,1)  
CONSTRAINT CHK_Puntuacion CHECK (Puntuacion BETWEEN 0.0 AND 10.0);
```

```
--b
```

```
ALTER TABLE Prestamo  
ADD Notas VARCHAR(300) NULL;
```

```
--C
```

```
ALTER TABLE Prestamo  
ADD CONSTRAINT CHK_Prestamo_Fechas  
CHECK (FechaDevolucionPrevista >= FechaPrestamo);
```

```
--Ejercicios
```

```
USE BibliotecaUniversitaria  
GO
```

```
--D1
```

```
SELECT l.titulo, l.isbn, c.nombre AS categoria  
FROM Libro l  
JOIN Categoria c ON l.id_categoria = c.id_categoria  
LEFT JOIN Ejemplar e ON l.id_libro = e.id_libro  
LEFT JOIN Prestamo p ON e.id_ejemplar = p.id_ejemplar  
WHERE p.id_prestamo IS NULL  
ORDER BY l.titulo;
```

```
--D2
```

```
SELECT  
c.nombre,  
COUNT(DISTINCT l.id_libro) AS libros,
```



```
COUNT(e.id_ejemplar) AS total_ejemplares,  
COUNT(CASE WHEN e.estado = 'Disponible' THEN 1 END) AS disponibles  
FROM Categoria c  
LEFT JOIN Libro l  
ON c.id_categoria = l.id_categoria  
LEFT JOIN Ejemplar e  
ON l.id_libro = e.id_libro  
GROUP BY c.nombre  
ORDER BY disponibles DESC;
```

--D3

```
SELECT  
s.nombre + ' ' + s.apellido AS socio,  
l.titulo,  
DATEDIFF(DAY, p.fecha_devolucion_prevista, p.fecha_devolucion_real) AS  
dias_retraso  
FROM Prestamo p  
JOIN Socio s ON p.id_socio = s.id_socio  
JOIN Ejemplar e ON p.id_ejemplar = e.id_ejemplar  
JOIN Libro l ON e.id_libro = l.id_libro  
WHERE p.fecha_devolucion_real > p.fecha_devolucion_prevista  
ORDER BY dias_retraso DESC;
```

--D4

```
SELECT TOP 3  
a.nombre + ' ' + a.apellido AS autor,  
a.nacionalidad, COUNT(*) AS total_prestamos
```

```
FROM Autor a
JOIN Libro_Autor la ON a.id_autor = la.id_autor
JOIN Libro l ON la.id_libro = l.id_libro
JOIN Ejemplar e ON l.id_libro = e.id_libro
JOIN Prestamo p ON e.id_ejemplar = p.id_ejemplar
GROUP BY a.nombre, a.apellido, a.nacionalidad
ORDER BY total_prestamos DESC;
```